中国研究生操作系统开源创新大赛

项目说明书

作 品 名 称：	VectorLease
参 赛 队 伍：	锐 s 刻五代
完 成 日 期：	2026 年 7 月

目 录

目 录 2
第 1 章 绪 论 3
1.1 目的与意义 3
1.2 项目背景 3
第 2 章 技 术 理 论 5
2.1 设计思想 5
2.2 技术路线 6
2.3 代码原创说明 7
第 3 章 项目介绍 7
3.1 功能介绍 7
第 4 章 实验测试 12
4.1 测试说明 12
4.2 测试结果 13
4.3 相位同步的直接验证：OpenMP 屏障到达实测 17
4.4 机制消融与竞争性解释排除 19
4.5 x86_64 跨架构补充验证 22
第 5 章 实现难点说明 22
5.1 sched_ext 向 K1 Linux 6.6 的移植 22
5.2 协同线程的可靠识别、分组与核映射 23
5.3 性能采集、归因与口径控制 23
5.4 用户态阶段提示与内核调度协同 23
第 6 章 总 结 24
6.1 项目完成情况 24
6.2 核心结论与贡献 24
6.3 局限与后续工作 24
6.4 开源项目致谢 25

第 1 章　绪 论

1.1　目的与意义

大语言模型（LLM）在边缘与端侧设备上的部署持续推进。受限于处理器算力与功耗，如何在这类平台维持推理吞吐、压低服务延迟，已成为计算机系统研究中的现实问题。CPU 侧推理引擎通常以线程池并发执行逐层张量计算图，同组线程在每层算子前后需经硬屏障对齐；单个 Token 生成中此类同步往往重复数百次。系统效率的决定因素并非单线程能否独占处理器，而是同组线程能否在屏障处近似同时到达。若到达时刻离散，先到者只能自旋或睡眠，整组节奏由最迟到达者决定。RISC-V 标准向量扩展（RVV；RISC-V International，2021）为张量计算提供 SIMD 加速，但 Linux 默认完全公平调度器（CFS）既不感知屏障协同关系，也不感知向量执行状态。在资源竞争下，调度器可能将同组线程分时复用至同一核心、跨核迁移或插入其他任务，使屏障相位发散，造成算力空耗、自旋与额外切换。针对上述问题，本作品以屏障相位为直接控制目标，设计面向 RISC-V LLM 推理的内核协同调度系统；并在 x86_64 AVX2 平台进行地位低于主线的跨架构补充验证。

为此，本作品基于 Linux 可编程内核调度框架 sched_ext（sched-ext project，2023；Linux Kernel Contributors，2024），构建以同相位推进为核心的调度系统。对于经识别的向量线程，调度器建立组内互斥的稳定 CPU 映射，使同一实例的线程分别独占核心、以相近节拍推进，并在屏障处同步到达。RVV 状态识别、专用向量调度队列（VDSQ）与 100 ms 向量时间片分别服务于身份鉴别与相位连续性维持；系统同时接收 Prefill、Decode 阶段提示，用于多实例竞争下的组间仲裁。

本作品的意义主要有三：（1）提出以屏障相位为目标的内核协同调度机制，并在标准 Linux 与 RISC-V RVV 平台给出可复现方案；（2）验证 sched_ext 在 AI 推理场景的可行性，拓展可编程内核调度的应用边界；（3）构建集相位测量、异步竞争负载与跨架构验证于一体的评测体系，为后续研究提供测试基础。

1.2　项目背景

主流 CPU 侧推理引擎（如 llama.cpp；ggml-org，2023）通过线程池并行执行计算图，各线程在每层算子前后借助 futex 屏障严格对齐。本文将线程到达屏障入口的时刻称为其屏障相位，将同组相位的离散程度称为到达时差。CFS 不掌握线程协同关系，而是将各线程视为独立实体，对其分时调度与负载均衡。

当某线程因时间片耗尽、被抢占或迁移而滞后时，其余线程往往已到达屏障并自旋或睡眠，整层完成时刻取决于最迟到达者。这种相位发散在核心充裕时会被较高 CPU 利用率掩盖，在多实例共置、线程数超过核心数时充分暴露。

近年来已有若干探索。Son（2026）的 Elastic Gang 在自研 Anima OS 中实现推理 gang 调度，出发点与屏障对齐最接近，但依赖自研系统与引擎。Xu 等（2026）的 ArcLight 在框架内重构 NUMA 感知调度，最高提升约 46% 吞吐，但决策位于应用层，屏障相位缺乏内核保证。Roca 与 Beltran（2026）的 SCHED_COOP 扩展 glibc 接管调度，过载测试最高提升约 2.4 倍，说明 CFS 在密集屏障负载下严重退化，但依赖修改 C 运行库，且未建模屏障相位。Gottschlag 等（2020）识别 SIMD 任务并隔离核心，目标是消除宽向量降频干扰。

综合现有工作，该领域仍有四点不足。其一，已有研究多面向 x86-64，针对 RISC-V 及 RVV 的调度优化尚未见系统性探索；x86 侧工作也未直接度量或控制屏障相位。其二，控制目标缺位：现有方案要么采用自研内核中的整体 gang 调度，要么停留于应用层启发式优化，均未在标准内核中将同组线程的屏障相位作为直接控制对象。sched_ext 允许以内核级控制力加载 BPF 策略并保持生态兼容，却尚未用于该目标。其三，缺乏对相位的直接度量：已有研究多借助切换次数或 CPU 时间间接推断协同质量，无法区分相位对齐与单纯计数变化，因而难以可靠归因。其四，用户态与内核态之间缺少低开销语义通道：框架掌握 Prefill/Decode 阶段，内核掌握任务就绪状态与处理器负载，二者的信息互通对组间仲裁仍然必要。

针对上述问题，本项目基于 sched_ext 实现面向 RISC-V LLM 推理的同相位协同调度系统：通过 RVV 脏状态识别与用户态注册鉴别向量线程，借助组内互斥的稳定核心映射、per-team/per-CPU VDSQ 与延长的向量时间片，使同一实例的线程以相近节拍推进、在屏障处同步到达；阶段提示仅在多实例竞争同一批核心时用于组间仲裁。微观测量表明，双实例 8+8 线程共享 8 核时，Decode 阶段同组线程的到达时差中位数由 CFS 下的 850.500 μs 降至 13.386 μs，95 分位由 3176.455 μs 降至 144.923 μs，降幅 95.4%。在输入 128 Token、输出 256 Token 的双模型异步负载中，2、4、8 并发下聚合吞吐相对 CFS 分别提升 54.9%、30.7% 与 25.6%，注入请求平均首 Token 延迟分别降低 39.0%、42.0% 与 42.2%，95 分位值分别降低 38.6%、42.3% 与 42.5%；上下文切换同步下降 93.1%～96.4%，这是相位对齐后自旋与重排减少的派生结果。具体设计与实验见后续章节。

第 2 章 技 术 理 论

2.1 设计思想

本项目采用测量驱动、假设验证的迭代式方法，沿“基线—假设—上界—控制变量—消融归因”闭环推进，核心测量屏障到达相位。每项机制以独立开关实现，与 CFS 同负载对比；在确认稳定性与完整性后多轮验证，综合吞吐、延迟与开销评估。

项目初期曾假设宽向量寄存器的保存与恢复是主要瓶颈。为此实现的 restore-elision 内核可跳过 v0-v31 恢复，完整恢复省略率达 93.6%、向量搬运减少约 48.4%，但 TPS 变化不足 0.1%（详见 4.4.1）。该负结果将优化方向指向线程间协同关系。

线程屏障协同对效率具有决定性影响。llama.cpp 通过 OpenMP 创建与核心数相等的持久线程，每层算子完成后均在屏障处同步再进入下一层，一个 Token 生成中重复数百次。futex 追踪表明，CFS 不掌握该协同关系，可能将同组线程分时调度至同一核心、插入其他任务或跨核迁移，使先到者反复经历短周期睡眠与唤醒。双实例共置时 16 线程竞争 8 核，调度器任务选择空间充分，相位发散随之暴露。机制部署前后全量 futex 累计阻塞工作时间基本不变，说明同步工作量未减，变化的只是阻塞分布：长周期离散停滞转为少量集中短等待。由此可确立核心假设：瓶颈在于屏障相位发散，而非同步指令开销。

围绕保护并锁定屏障相位，机制分三层组织。首先，调度器在线程切出时检测 sstatus.VS=Dirty，经阈值确认线程身份，并结合 llama.cpp 经 mmap 注册的信息辅助鉴别。其次，调度器为同一实例建立组内互斥的稳定 CPU 映射，从空间上消除同组线程在同一核心串行化的可能；同时分配 100 ms 向量时间片，避免两层屏障间的连续计算被常规时间片打断。最后，引入 per-team/per-CPU 专用 VDSQ，普通任务与其他实例线程无法随意插入执行窗口，每个 CPU 仍保留 global DSQ。阶段性原型在 500 ms 本地租约窗口下，将单实例 TPS 提升约 0.9%、TTFT 降低约 14%，将双实例共置 TPS 提升 19.1%。

本文将上述执行状态定义为同相位调度：同一实例的线程各自占用互不重复的 CPU，在计算图上以相近速度推进，屏障入口时刻趋于一致，即到达时差趋零的锁步状态。此时最迟到达者不再因同核竞争、任务插入或迁移而滞后。与单纯调整时间片不同，同相位的本质是让每个线程稳定回到映射核心、按相同相位推进；100 ms 时间片与 VDSQ 隔离只是维持该绑定的手段。

探索过程中还实现了按计算图边界动态授权的路线：patched llama-server 在图边界通过用户态控制器授予 OpenMP team 宽度，再由 sched_ext 将授权线

程放入自定义 DSQ。补充短测（见 4.4.3）显示，该路线切换最少，但吞吐与同相位调度持平，约为 1.85 与 1.86 TPS。这说明单纯压缩切换并非性能来源；同相位调度无需授权握手，侵入更小。

稳定核心映射解决实例内部相位问题。多实例共置时，新请求以计算密集的 Prefill 开始，与正在 Decode 的线程竞争 CPU，可能拉长首 Token 延迟。为此 llama.cpp 发布 Prefill、Decode、Mixed 与 Idle 四种提示，调度器优先选择 Prefill/Mixed 实例的 VDSQ；仅当优先线程唤醒、目标核心正运行另一组向量线程时定向抢占。该机制属于组间仲裁并与同相位推进解耦：Observe 仅具备同相位、不改组间队列，已取得主要吞吐收益；阶段感知在吞吐基本不变时，使平均 TTFT 再降低 12.9%、21.5% 与 13.7%。

软件架构遵循低侵入、分层协同与模块解耦。BPF 程序负责任务识别、CPU 分配、VDSQ 与决策；loader 负责加载配置；llama.cpp 仅增加注册与提示；观测工具独立采集数据。各模块经带版本校验的 BPF map 通信，可独立替换。

2.2 技术路线

本项目采用“Linux sched_ext/eBPF 可编程调度、llama.cpp 用户态语义提示、RISC-V Vector 状态感知”相结合的框架。内核侧提供任务就绪状态、CPU 执行状态与队列控制能力，用户侧提供线程身份与计算阶段语义；两侧通过 BPF map 交换信息，最终决策由内核完成。

内核侧以移植到 K1 厂商 Linux 6.6 的 sched_ext 为基础，通过 BPF struct_ops 暴露 select_cpu、enqueue、dispatch、running 与 stopping 等回调。策略以 CO-RE eBPF 程序实现，由 libbpf 运行时加载；目标内核 BTF 提供结构类型信息，使其能读取 task_struct 中的 RVV 状态，并使用 DSQ、CPU kick 等原语。BPF 策略退出或异常后，任务重新交由默认调度器管理。

用户侧，patched llama.cpp 与 mmap 化 BPF map 构成低开销语义通道。CPU 后端注册线程 TID、TGID 与逻辑序号 ith，continuous batching 路径发布四类阶段；高频状态经 mmap 写入 workers map，loader 经 config map 完成加载、初始化与在线控制。接口不可用或校验失败时，llama.cpp 保持原有流程，调度器回退至 RVV 脏状态识别。

核心策略为 Phase-Aware Dirty Lease。eBPF 程序结合用户注册与 sstatus.VS=Dirty 识别向量线程，为同一实例建立组内互斥的稳定映射以实现同相位推进，并通过 per-team/per-CPU VDSQ 组织两组任务；local/global DSQ 保留本地执行与普通任务路径。调度器依据阶段提示优先选择 Prefill/Mixed 实例，并通过受限抢占与 pending safety 兼顾首 Token 响应与任务活性（见

3.1.2 节）。

作为跨架构补充，系统还移植至 x86_64 AVX2 平台。该平台运行内置 sched_ext 支持的 Linux 6.18 内核，无需厂商补丁；相位同步的基本单位由逻辑核调整为物理核，线程身份改由用户态 ABI worker map 鉴别，任务放置依据物理核与 sibling 关系安排（见 3.1.5 节）。

2.3 代码原创说明

sched_ext 核心框架源自 sched-ext/scx-kernel-releases 项目 v6.6.4-scx2 标签，经 2 项平台级移植补丁完整集成，包含 kernel/sched/ext.c（4510 行）、include/linux/sched/ext.h（716 行）及 kernel/sched/ext.h（269 行），共 5495 行（版权：Meta Platforms / Tejun Heo / David Vernet，GPL-2.0）。

推理引擎语义透传基于 llama.cpp 基线 678d43d7 拓展，涉及 11 个源文件（7 修改 + 4 新增），变更 +897 / -6 行（MIT，版权：The ggml authors）。

scx_llm_pin 依赖 libbpf（源自内核 tools/lib/bpf/，LGPL-2.1 OR BSD-2-Clause）。

观测组件 rvv_sched_cost 基于 libbpf，静态链接 libelf（elfutils）。

vmlinux.h（各 180639 行）由 bpftool 自开发板运行内核 BTF（/sys/kernel/btf/vmlinux）生成，版权属于 Linux 内核社区。

x86 补充部分含 BPF 策略 1327 行、loader 748 行及 ABI/拓扑定义，观测器含 BPF 端 288 行与用户态 277 行；复用同一 patched llama.cpp 接口，不含 K1 内核补丁

第 3 章 项目介绍

3.1 功能介绍

本系统是面向 RISC-V CPU LLM 推理负载的可编程内核调度软件，运行于已移植 sched_ext 的 K1 Linux 6.6 内核。系统通过 patched llama.cpp 获取线程身份与阶段语义，通过 eBPF 调度器读取内核任务状态并实施 CPU 调度。其核心是以稳定核心映射实现同组线程的同相位推进，并辅以 RVV 识别、分组队列、阶段优先、在线控制、状态观测与端到端测试。

系统采用“用户态语义发布、共享状态通信、内核态决策、Linux 核心执行”的分层架构。llama-server、loader 与控制工具不直接操作运行队列，而是通过 pinned BPF maps 与 struct_ops 协作；调度器卸载或异常后，任务回退默

认调度器。总体通信关系如图 3-1。

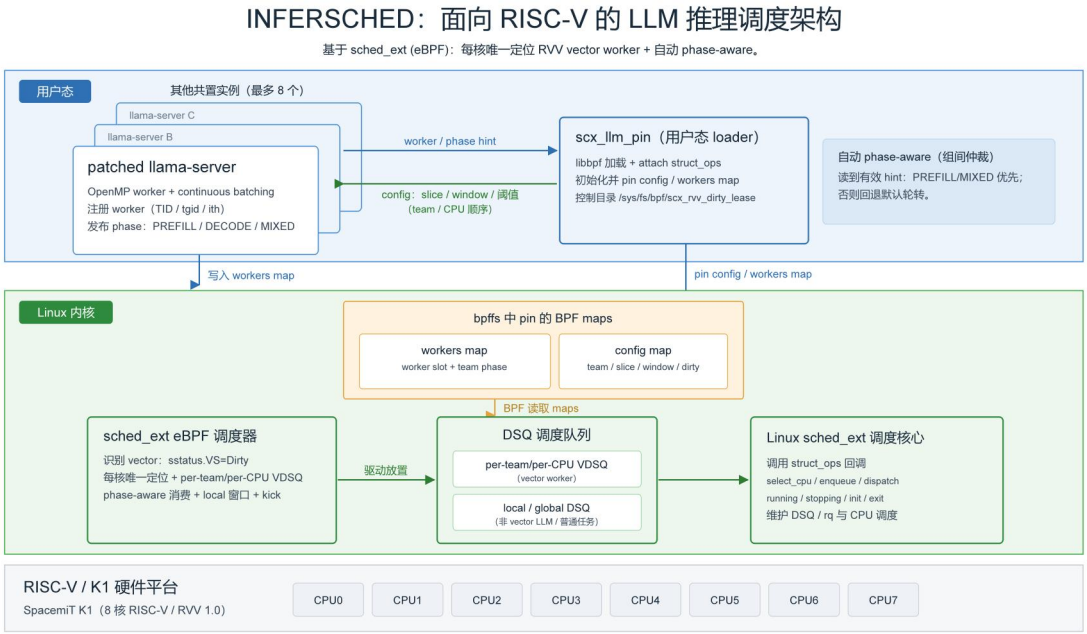


图 3-1 RVV-aware sched_ext 用户态与 Linux 内核通信架构

按功能与数据流，系统分为用户态语义与控制、Phase-Aware Dirty Lease 调度、Linux sched_ext 执行以及观测与测试四个模块。

3.1.1 用户态语义与控制模块

该模块由 patched llama-server、llama_context 与 GGML CPU 后端组成。OpenMP 线程启动后注册 TID、TGID、逻辑序号 ith 及所属实例；推理中根据 batch 内 prompt token 与 generation token 组成发布 Prefill、Decode、Mixed 或 Idle 阶段。信息经 mmap 直接写入 workers map，不改变计算、采样与请求逻辑；控制 map 缺失、版本不匹配或 sched_ext 未启用时，接口退化为空操作，同一二进制仍可运行于默认调度器。

workers map 保存线程身份、ith、team、phase、epoch 与会话状态，config map 保存时间片、窗口、阈值、CPU 映射与开关；高频语义经 mmap 更新，低频配置经系统调用读写，版本与会话校验阻止失效提示。scx_llm_pin loader 负责加载 CO-RE object、初始化 pin maps、写入参数并附加 struct_ops；rvv_lease_ctl 运行期在线读取与调整，退出时卸载并清理。

3.1.2 Phase-Aware Dirty Lease 调度模块

该模块是系统核心，不创建执行实体，而是对 sched_ext 管理的就绪任务分类、入队与选取。每个 CPU 拥有一个内核 local DSQ，两组 llama-server 分别拥有 VDSQ(A,i) 与 VDSQ(B,i)；图 3-2 仅展示 CPU i，相同结构在 CPU0～CPU7 重复。。

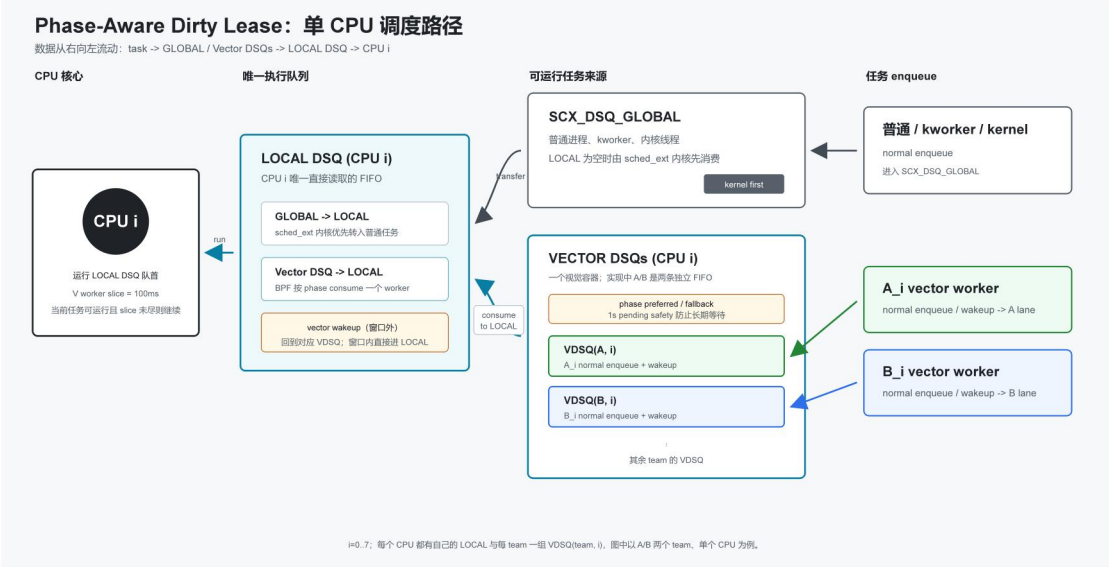


图 3-2 Phase-Aware Dirty Lease 单 CPU 调度路径

CFS 在该负载下的相位问题有三类：同组线程被分时复用至同一核心而串行化、普通任务插入连续计算窗口，以及他实例线程挤占核心而使本组并行宽度缩水。六个步骤分别针对上问题理设计。

（1）识别线程并建立稳定映射。BPF 先依据 TGID 判断 team，再经 workers map 中的 TID/ith 确认 OpenMP 线程；提示不可用时，以累计命中 sstatus.VS=Dirty 回退识别。调度器优先使用 ith→CPU 映射，否则为该 team 原子分配空闲 CPU。两组线程可共享 CPU0～CPU7，但同组保持唯一映射，这是同相位推进的前提；启动阶段允许暂时 direct-local，识别完成后统一进入 VDSQ。

（2）按类型入队。普通进程、辅助线程与内核线程进入 SCX_DSQ_GLOBAL；向量线程获得 100 ms 时间片，并按 team 与映射 CPU 进入对应 VDSQ。普通唤醒与时间片耗尽后均返回所属 VDSQ，同时记录 pending 起始时间。

（3）Linux 核心执行。CPU 只运行 local DSQ；local 为空时先服务 global

DSQ，仍无任务才调用 BPF dispatch，从该 CPU 的两条 VDSQ 转出一个线程。当前任务在可运行且时间片未耗尽时继续，阻塞、让出或耗尽后才重新选择。

（4）依据阶段选择 VDSQ。通过校验的 Prefill/Mixed 实例在另一组不延迟敏感时成为 preferred team，优先消费其 VDSQ；首选队列为空则立即回退。两组阶段相同或都延迟敏感时，按 per-CPU next-team 轮换。每次 dispatch 仅转出一个线程，保持 work-conserving。

（5）处理延迟敏感唤醒。preferred 线程唤醒且目标 CPU 正运行另一组向量线程时，才以 SCX_ENQ_PREEMPT direct-local 请求抢占；同 team、普通入队、时间片耗尽与非向量任务均不触发。

（6）维持活性。线程在屏障处执行 futex wait 时立即释放 CPU，stopping 清除 running-team 状态；向量线程连续等待超 1 s 后，pending safety 优先消费等待最久的 team。阶段失效、首选队列为空或线程睡眠时，均回退其他可运行任务。

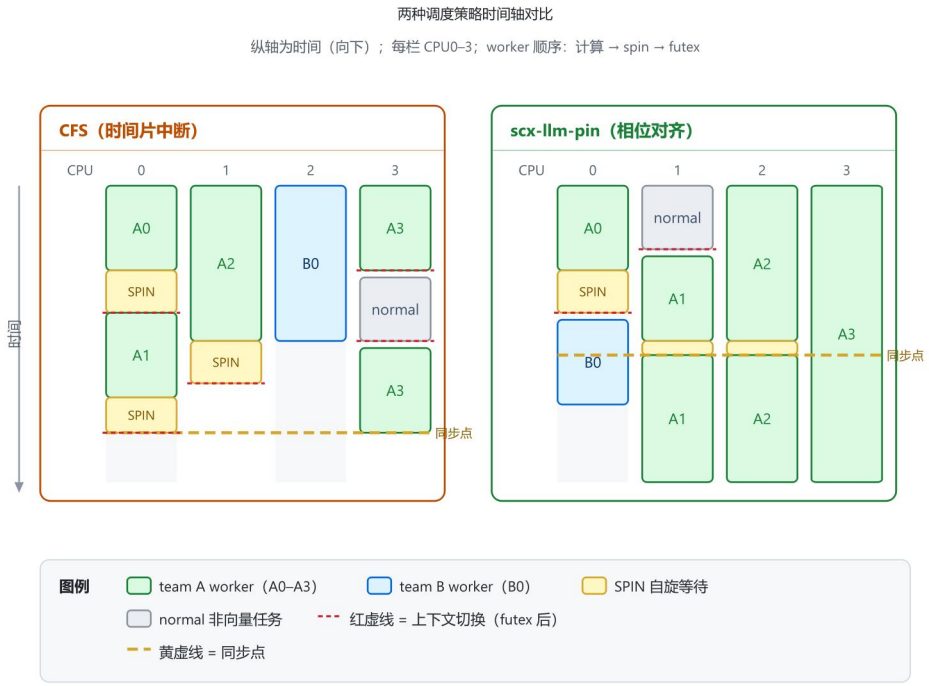


图 3-3 CFS 与同相位调度的逐核时间轴对比

图 3-3 以 8 核平台中的 4 个核心与 team A 的 4 个线程为示意，逐核对比两类调度结果。左栏 CFS 的同步点被拖至末端，各核问题为：CPU0 上，A0 完成分片后自旋，于 futex 后切出；同组 A1 只能等待 A0 让出后，才在同一核心开始计算，两个协同线程因核心复用被串行化，A1 的相位凭空推迟一个轮转。CPU1 上，A2 独占核心且较早完成，却因他核线程未到而持续自旋，先到者承担

等待代价。CPU2 整个窗口被他实例 B0 占用，team A 实际损失一个核心，组内并行宽度由 4 退化为 3。CPU3 上，A3 被 normal 任务两次插入、割裂为三段，最终沦为最迟到达者，全组同步点由其相位单独决定。

右栏给出对应修复。针对 CPU0 核心复用，稳定映射将 A1 固定至 CPU1，使其立即开始连续计算；针对 CPU3 插入打断，VDSQ 隔离与 100 ms 时间片使 A3 连续运行，普通任务只在窗口前部短时运行；针对 CPU2 他组挤占，per-team VDSQ 使 B0 仅在 A0 睡眠后填充 CPU0，不抢占正在计算的线程。于是 A1、A2、A3 同步到达屏障，全组对齐时刻提前。图 3-3 的结论将在 4.3 节定量验证。

3.1.3 Linux sched_ext 执行模块

该模块由移植至 K1 Linux 6.6 的 sched_ext 调度类提供。Linux 核心调用 eBPF 实现的 select_cpu、enqueue、dispatch、running、stopping 等回调，并负责 DSQ、运行队列、CPU 运行与实际切换。任务从目标 CPU 的 local DSQ 运行；local 为空时先服务 global DSQ，再经 BPF dispatch 从 VDSQ 补充。退出与 watchdog 机制提供统一回退边界，策略退出后任务重新由默认调度类管理。

3.1.4 观测与测试模块

该模块与调度路径解耦。`rvv_sched_cost` 使用 CO-RE eBPF 采集切换、RVV save/restore、wake-to-run 与 futex 数据；屏障探针 `libbarrier_probe.so` 直接测量到达时差（见 4.3 节）。Benchmark 负责启停、预热、加载、流式请求、采样、汇总与恢复，记录 TPS、逐请求 TTFT、pooled p95 TTFT、公平性与切换，并对请求完整性、线程数、DSQ 收支、频温、内存与 watchdog 进行检查。

3.1.4 x86 SMT 相位适配

作为跨架构补充，x86_64 版沿用同一机制并针对同步多线程作两点调整。其一，x86 无架构级向量脏状态位，工作线程分类改由 ABI worker map 完成；内核校验 TID、TGID、序号与实例后建立映射，该平台无 RVV 式硬件回退识别。其二，相位同步以物理核为单位：在 4 物理核、8 逻辑 CPU 平台上，单实例占四个物理主核；Prefill 与 Decode 重叠时，两组均映射至主核，由阶段仲裁优先 Prefill，避免 sibling 竞争 AVX2 执行资源；两组均进入 Decode 后，其中一组才转用 sibling。加载器附加前校验拓扑，防止错误映射破坏相位前提。该模块不改变策略结构，仅适配映射层次。

第 4 章 实验测试

4.1 测试说明

4.1.1 测试环境

实验环境见表 4-1。各策略使用相同模型和 llama-server，仅切换 CFS 与 sched_ext；CPU 固定为 1.6 GHz，Swap 关闭。

表 4-1 测试环境

类别	配置
开发板	Milk-V Jupiter
处理器	SpacemiT K1，8 核 RISC-V，支持 RVV 1.0
内存	15.5 GiB，Swap 关闭
操作系统	Bianbu Linux，K1 厂商 Linux 6.6.63+，移植 sched_ext
CPU 频率	performance governor，固定 1.6 GHz
推理引擎	patched llama.cpp，基线提交 678d43d7205cffc3a8a026cb60b1de3868cc70a2
模型	Llama-3.2-3B-Instruct-Q4_K_M.gguf
服务配置	两个独立 llama-server，各使用 8 个计算线程，上下文长度 4096
调度程序	scx_llm_pin，CO-RE eBPF struct_ops 调度器

4.1.2 测试负载与对照策略

实验采用双模型异步竞争：两个 8-worker llama-server 共享 CPU0～CPU7。每轮先向 A 发送 incumbent 请求，均输出第 4 个 Token 后，再以 1 s 间隔向 B 注入等量请求；c=2、4、8 分别对应两侧 1∶1、2∶2、4∶4。该时序使 B 的 Prefill 与 A 的 Decode 重叠，可同时测量吞吐与竞争延迟。Prompt 无共同长前缀，统一补齐为 128 input token；每请求固定输出 256 Token，temperature=0、seed=1234、cache_prompt=false，预热后经流式接口记录逐

Token 时间。

三策略见表 4-2。Observe 具备同相位但不依据提示改变队列选择，完整策略额外启用组间仲裁，故 CFS→Observe 度量同相位机制，Observe→Phase-Aware 度量阶段感知增量。

表 4-2 对照策略与关键配置

策略	调度配置	实验作用
CFS	Linux 默认公平调度，两个服务共享 8 核	基线
Dirty Lease Observe	dirty threshold=2，vector slice=100 ms，window=500 ms，vector-local-wakeup=off，读取 hint 但不改变队列选择	消融阶段感知机制，评价基础延长时间片策略
Phase-Aware Dirty Lease	在 Observe 配置上启用 prefill-lease、1 s pending safety 和 Prefill wakeup 定向抢占	完整策略

4.1.3 指标与统计方法

Aggregate TPS 按一轮实际 Token 总数除以真实 makespan；TTFT 为发送至首 Token 到达的时间，仅统计注入 B 的请求；p95 取全部注入样本的 nearest-rank 结果。每策略在 c=2、4、8 下分别执行 60、30、15 个有效轮次，均获 60 个注入样本。TPS 按轮统计均值与标准差，TTFT 按 pooled 单请求统计。切换与自旋时间为机制指标，不作为关键路径收益的独立判据。

4.2 测试结果

4.2.1 吞吐、首 Token 延迟与调度开销

表 4-3 汇总 c=2、4、8 时的 Aggregate TPS、注入请求平均 TTFT 和 p95 TTFT。括号内为相对同并发 CFS 的变化。

表 4-3-1 c=2 实验性能结果

策略	**Aggregate TPS（均值 ± 标准差）**	**注入 TTFT（均值 ± 标准差）**	**注入 TTFT p95**
CFS	1.6300 ± 0.0340	22.786 ± 0.291 s	23.195 s
Dirty Lease Observe	**2.5347 ± 0.0227（+55.5%）**	15.956 ± 0.483 s（-30.0%）	16.747 s（-27.8%）
Phase-Aware Dirty Lease	2.5250 ± 0.0168（+54.9%）	**13.904 ± 0.181 s（-39.0%）**	**14.253 s（-38.6%）**

表 4-3-2　c=4 实验性能结果

策略	**Aggregate TPS（均值 ± 标准差）**	**注入 TTFT（均值 ± 标准差）**	**注入 TTFT p95**
CFS	2.0701 ± 0.0244	36.513 ± 12.556 s	49.724 s
Dirty Lease Observe	2.6995 ± 0.0176（+30.4%）	26.975 ± 9.353 s（-26.1%）	37.767 s（-24.0%）
Phase-Aware Dirty Lease	**2.7056 ± 0.0122（+30.7%）**	**21.173 ± 7.076 s（-42.0%）**	**28.677 s（-42.3%**

表 4-3-3　c=8 实验性能结果

策略	**Aggregate TPS（均值 ± 标准差）**	**注入 TTFT（均值 ± 标准差）**	**注入 TTFT p95**
CFS	3.2008 ± 0.0288	76.130 ± 30.395 s	96.264 s
Dirty Lease Observe	4.0109 ± 0.0195（+25.3%）	50.988 ± 20.191 s（-33.0%）	65.434 s（-32.0%）

策略	Aggregate TPS（均值 ± 标准差）	注入 TTFT（均值 ± 标准差）	注入 TTFT p95
Phase-Aware Dirty Lease	**4.0210 ± 0.0080（+25.6%）**	**43.983 ± 17.556 s（-42.2%）**	**55.395 s（-42.5%）**

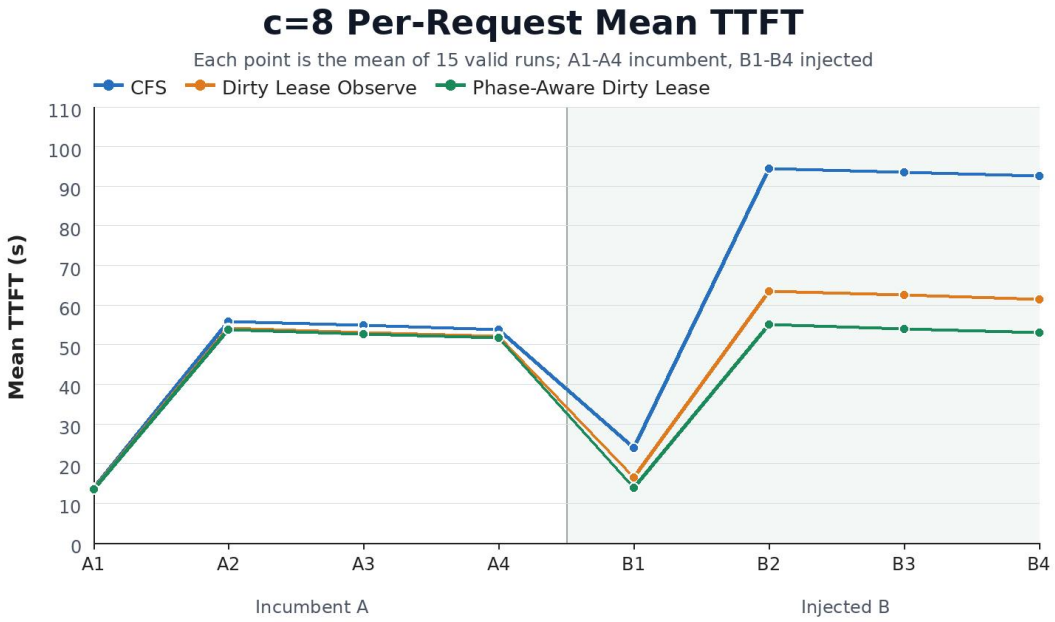


图 4-1　c=8 逐请求平均 TTFT

图 4-1 按请求槽位展示 c=8 的 15 轮平均 TTFT，其中 A 为 incumbent 请求，B 为后注入请求。

图 4-2　c=4 B2 请求 TPS 时间曲线

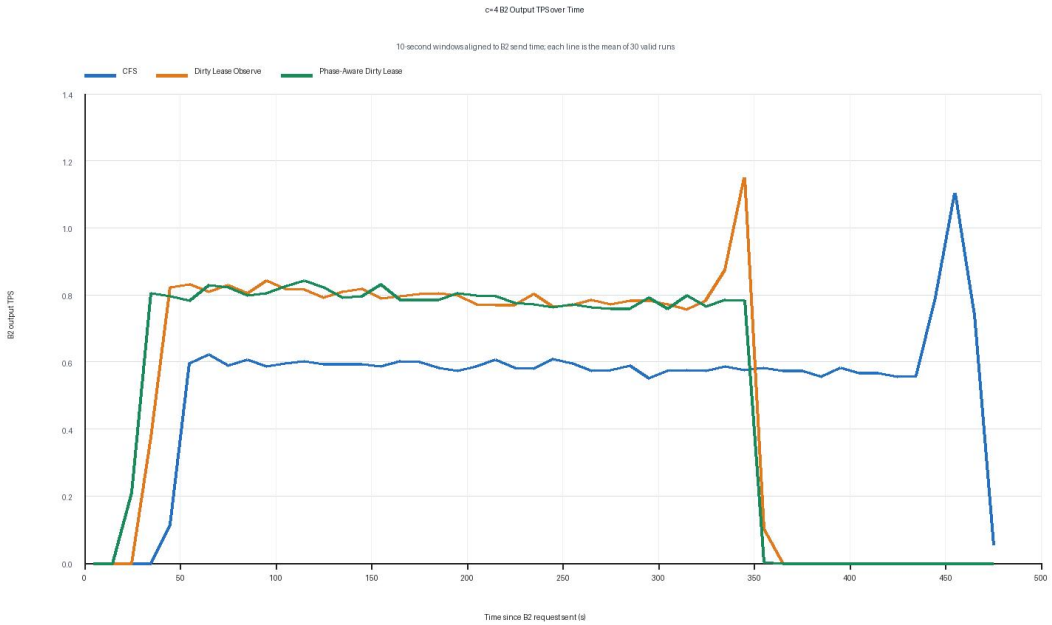


图 4-2 展示 c=4 时 B2 的 30 轮平均窗口 TPS。Phase-Aware 与 Observe 完成时间接近，但前者更早输出，说明阶段感知主要提前 Prefill 的运行机会。

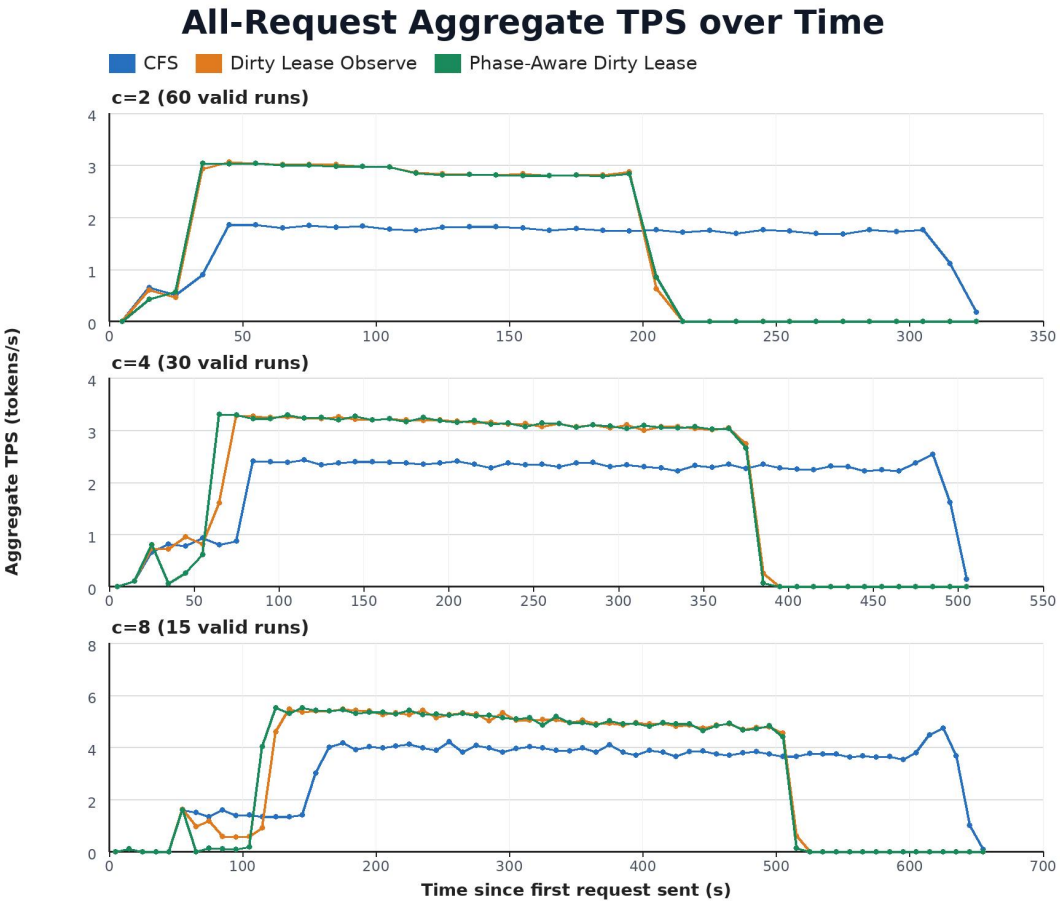


图 4-3　三种策略全请求 Aggregate TPS 时间曲线

图 4-3 以每轮首个请求为时间零点，按 10 s 窗口展示两个服务的平均 Aggregate TPS；其统计对象与图 4-2 不同，横坐标不直接对应。

相对 CFS，Phase-Aware 在三种并发下将 TPS 提高 25.6%～54.9%，平均和

p95 TTFT 分别降低 39.0%～42.2% 和 38.6%～42.5%，TPS 跨轮变异系数不超过 2.1%。Observe 已取得主要吞吐收益；Phase-Aware 相对 Observe 的 TPS 变化不超过 0.4%，但平均和 p95 TTFT 进一步降低 12.9%～21.5% 和 14.9%～24.1%，说明阶段提示主要调整 Prefill 与 Decode 获得资源的顺序，而非增加总计算能力。

表 4-4 给出两个 llama-server 进程组在单轮测试中的通用上下文切换总数。

表 4-4　双模型上下文切换次数（均值 ± 标准差）

总并发数	CFS	Dirty Lease Observe	Phase-Aware Dirty Lease
2	1,436,746 ± 65,273	52,394 ± 1,508（-96.4%）	**51,462 ± 1,652（-96.4%）**
4	1,651,506 ± 41,296	60,039 ± 803（-96.4%）	**59,984 ± 798（-96.4%）**
8	1,995,704 ± 41,305	138,141 ± 2,628（-93.1%）	**137,692 ± 3,357（-93.1%）**

完整策略相对 CFS 减少 93.1%～96.4% 的上下文切换。Observe 与其结果接近，说明切换下降主要来自稳定映射、VDSQ 和长 slice，而 Phase-Aware 的额外延迟收益来自组间顺序调整。

4.2.2　测试结论

完整测试矩阵共包含 315 个有效轮次，实验验证了系统的两层作用：Dirty Lease 基础机制主要提升吞吐并大幅减少上下文切换，Phase-Aware 机制则利用用户态阶段提示，在保持吞吐的同时进一步降低新请求的平均与尾部 TTFT。

4.3　相位同步的直接验证：OpenMP 屏障到达实测

本节以屏障探针直接测量，对图 3-3 的定性预测作定量检验。。

4.3.1 测量机理与指标定义

libbarrier_probe.so 通过动态符号插入包装显式 GOMP_barrier，原函数经 RTLD_NEXT 调用，原版 libgomp 的自旋与睡眠机制保持不变；探针经--preload 注入，不影响启动器与其他进程。每个 team 固定 8 个持久线程，按线程独立递增的显式屏障序号配对；每个 64-barrier 区块以共同散列选择一个采样位置，避免固定模数与算子周期重合，每轮换 seed。采样线程在调用原屏障前后各读一次 CLOCK_MONOTONIC，各写独占的 64 字节缓存行，不存在观测器侧全组锁；预触页、身份初始化与预热均在正式窗口外完成。

本节采用两个直接指标：到达时差（arrival skew）为同一 epoch 内最晚与最早入口之差，刻画组内相位离散度；释放尾时延（release tail）为最晚入口至全组返回之差，刻画最后一个线程到达后的同步解除代价。测量覆盖显式 GOMP_barrier 与 ggml 显式图同步，不覆盖 parallel region 结束等隐式屏障；Decode 样本按各请求首 Token 时间划分，边界前后 1 ms 排除。完整性门禁包括 8 个不同 TID、各线程序号一致、每 epoch 8 入口 8 返回、返回不早于最后入口等。

4.3.2 测量结果与分析

三个对照组使用同一 llama-server、libgomp 与模型，8+8 线程共享 CPU0～CPU7，负载为 128 input / 64 output。CFS 使用默认时间片；scx 为完整策略，使用 100 ms 时间片、500 ms 窗口与阶段提示；scx-short 额外设置 GOMP_SPINCOUNT=1000，作为全局短自旋控制。每组含两个探针轮与一个无 preload 的 plain 轮，以 plain 轮吞吐估计探针扰动。结果见表 4-5，数值为有效轮均值。

表 4-5 屏障到达特征实测（Decode 阶段，μs）

策略	探针轮次	TPS，探针/plain	到达时差均值/中位数/p95	释放尾时延均值
CFS	2	1.2756 / 1.2476	1027.970 / 850.500 / 3176.455	1404.863
scx	2	1.9805 / 1.9930	240.456 / 13.386 / 144.923	5901.092
scx-short	2	1.7964 / 1.8009	485.658 / 120.533 / 2058.728	649.433

结果直接证实同相位状态。Decode 阶段到达时差均值由 1027.970 μs 降至 240.456 μs，降幅 76.6%；中位数由 850.500 μs 降至 13.386 μs，95 分位由 3176.455 μs 降至 144.923 μs，降幅 95.4%。结合图 3-3 可判断，同组线程在绝大多数 epoch 中已在十余微秒窗口内汇聚，CFS 下由同核竞争、任务插入与他组挤占造成的相位发散，已被稳定映射与队列隔离基本消除。探针轮与 plain 轮 TPS 差异很小，结论不来自插桩扰动。

释放尾时延提供进一步机制信息。scx 默认自旋使全组返回延至 5901.092 μs；scx-short 虽将其降至 649.433 μs，却使时差均值回升至 485.658 μs、TPS 降至 1.7964。这说明过早让核以相位发散为代价，总体效率由到达时差主导。

4.4 机制消融与竞争性解释排除

4.4.1 RVV 寄存器搬运开销的排除

为检验寄存器现场搬运是否为主要瓶颈，项目实现 restore-elision 定制内核：owner 状态有效时跳过 v0-v31 恢复，保留 eager save 与向量 CSR 恢复。实验使用单 llama-server、c=4、每轮 300 s，原厂与定制内核各 3 轮，CPU 固定 1.6 GHz。结果见表 4-6。

表 4-6 CFS 与 CFS + restore-elision 对比

配置	TPS（均值 ± 标准差）	平均 TTFT（均值 ± 标准差）	p95 TTFT	RVV save+restore 候选	实际完整 RVV save+restore
原厂 CFS	5.671 ± 0.092	3317.612 ± 209.135 ms	14264.225 ms	124,660 ± 2,245	124,660 ± 2,245
CFS + restore-elision	5.673 ± 0.070	3090.600 ± 381.757 ms	13269.032 ± 2006.988 ms	128,164 ± 1,906	69,758 ± 949
相对变化	+0.041%	-6.843%	-6.977%	+2.811%	-44.041%

定制内核省略 93.637% 完整恢复、减少 48.354% 数据搬运，但 TPS 仅提升 +0.041%，不具统计显著性。该结果将寄存器搬运排除出主要瓶颈，并与 2.1 节互证。

4.4.2 libgomp 自旋开销归因

双服务 c=8 下，各取 CFS、CFS + affinity 与 Observe 一轮样本用于归因。perf stat 统计系统指令数，cpu-clock:u 采样 on-CPU 符号，Build ID 与反汇编确认热点为 libgomp 进入 futex 前的 bounded-spin；受 K1 PMU 限制，符号占比仅表示 CPU 时间。

表 4-7 CFS、affinity 与 Observe 的 libgomp 自旋对比

配置	Aggregate TPS	LLM CPU runtime	系统指令数	spin-loop 用户态占比	spin-loop CPU 时间	libggml-cpu 采样数
CFS	2.1131	1799.24 s	2.000 T	17.54%	311.14 s	142,838
CFS + affinity	2.4177（+14.42%）	1605.23 s（-10.78%）	1.619 T（-19.05%）	7.48%	119.07 s	144,339（+1.05%）
Dirty Lease Observe	2.5194（+19.23%）	1528.83 s（-15.03%）	1.549 T（-22.55%）	5.64%	86.03 s	140,976（-1.30%）

Observe 将 spin-loop CPU 时间减少 72.35%，矩阵采样基本不变。这与 4.3 直接测量互证：自旋下降是时差压缩、线程近似同时到达后的结果，有效张量计算量未变。

4.4.3 动态授权对照补充短测

为对照另一路线，项目实现图边界动态授权 dynamic：patched llama-server 在图边界经用户态控制器 dt_controller 动态授予 OpenMP team 宽度，控制器利用 sched_ext 将授权线程放入 custom DSQ。该短测为 128 input / 64 output、concurrency=2 的同步场景，与 4.2 正式 campaign 口径不同，不并入表 4-3、4-4。

表 4-8 双模型同步短测（c=2、64 output、各 3 轮）

策略	**上下文切换（均值±标准差）**	**AggregateTPS（均值±标准差）**
CFS	381,980 ± 25,098	1.3327 ± 0.0367
CFS + 绑核 + 100 ms 长片 + 短 spin	564,942 ± 7,514	1.7527 ± 0.0122
dynamic，图边界动态授权	5,223 ± 576	1.8490 ± 0.1197
同相位调度，scx-llm-pin	13,318 ± 236	1.8582 ± 0.0155

dynamic 的切换仅为同相位调度约 40%，但两者吞吐相差约 0.5%。该对照

排除“切换下降驱动性能”的解释，与 4.4.2 共同支持本文归因：收益来自稳定映射下的相位对齐与资源分配，而非切换计数本身；同相位调度还免去授权与回收握手，侵入更小。

4.5 x86_64 跨架构补充验证

本节在 x86_64 AVX2 平台补充验证相位同步观点。结果不与 RISC-V 正式 campaign 合并，仅检验机制在不同拓扑下是否成立。

4.5.1 测试环境

平台为 Intel Core i5-10200H，4 物理核、8 SMT 逻辑 CPU，sibling 对为 0/4、1/5、2/6、3/7，支持 AVX2、FMA、XSAVE，无 AVX-512；系统为内置 sched_ext 的 Linux 6.18。模型为 Llama-3.2-3B-Instruct-Q4_K_M，每个 llama-server 4 计算线程。

4.5.2 非超售阶段优先与人工超售屏障实验

非超售场景，即 8 worker 共享 8 逻辑 CPU，无结构性冲突，组内相位对齐的吞吐空间有限，收益主要来自组间仲裁。单轮方向性验证显示，SMT 自适应使 TTFT 由约 5.03 s 降至 2.95 s，降幅 41.4%，TPS 仅下降 2.6%，与主线两层结论一致。

为构造等效相位冲突，两个 4 线程实例同时限制在 CPU0～CPU3，形成 8 worker 竞争 4 逻辑 CPU。CFS 无法协调两个 OpenMP 运行时，bounded-spin 占比 86.09%，TPS 仅 1.2384；启用稳定映射、分组 DSQ 与长 slice 后，spin 降至 5.36%，TPS 升至 8.1719，约 5.6 倍，makespan 缩短 84.8%，同步调用总数不变。核心归因在另一架构上复现。

配对观测显示 XSTATE 搬运不能解释该变化：切换减少约四分之三，但 save/restore 累计仅约 16.5 ms，占约 60 s 负载的 0.028%。结合 RISC-V 消融可知，两平台共同瓶颈均为屏障相位发散而非状态搬运。

第 5 章 实现难点说明

5.1 sched_ext 向 K1 Linux 6.6 的移植

K1 厂商 Linux 6.6 缺少 Linux 6.12 才进入主线的 sched_ext，且含独立

的调度、cgroup、BTF 与驱动修改，无法套用主线。项目以 `v6.6.4-scx2` 为基础,将调度类、struct_ops、DSQ 与 watchdog 接入厂商内核,完成 BTF/CO-RE、头文件与驱动适配。移植保留默认回退与控制边界：策略仅使用 DSQ、kick 等标准接口，退出或触发 watchdog 后释放任务。系统经内核构建、反复 attach/detach、双模型负载与 dmesg 门禁；补丁面向特定 K1 基线。

5.2 协同线程的可靠识别、分组与核映射

Linux 只见独立 task，`sstatus.VS=Dirty` 可表明线程使用过向量指令，却不能提供 team、序号与阶段；纯用户注册又有启动时序、版本失配与 TID 复用问题。项目采用两级识别：llama.cpp 发布 TID、TGID、ith、team 与会话，内核校验版本与会话后确认身份，并按 ith 建立同组唯一 CPU 映射，这是同相位推进的基础；提示未就绪时以累计 Dirty 保守回退，陈旧状态不参与决策。x86 无等价脏位，识别仅依赖 ABI 注册；映射须按物理核建立，加载器附加前校验 SMT 拓扑，并按阶段在主核与 sibling 之间切换放置，否则 AVX2 资源竞争会抵消对齐收益。

5.3 性能采集、归因与口径控制

请求时间、worker 事件与系统计数分属不同对象与时钟，观测本身也会扰动短周期调度。Benchmark 以单调时钟记录发送与逐 Token 时间,按真实 Token 与 makespan 计算指标；rvv_sched_cost 按 TGID/TID 关联 sched_switch、futex 与 RVV 事件；屏障探针以无锁、独立缓存行直接测量时差。符号分析针对 K1 PMU 限制采用低频 `cpu-clock:u`、Build ID 与反汇编确认热点。测试分离正式轮与诊断轮，区分切换、RVV 候选、实际搬运、futex blocked time 与自旋，避免将计数下降直接视为收益。正式结果固定口径、pooled p95，并通过完整性、DSQ 收支、频温与内核门禁。

5.4 用户态阶段提示与内核调度协同

Prefill、Decode、Mixed 与 Idle 仅框架掌握，而运行队列只能由内核安全控制，跨层接口须让用户态表达意图、内核保留决策权。项目在 continuous batching 与 GGML 后端加入低侵入接口，经 mmap 更新身份与阶段，高频路径不经 BPF 系统调用；loader 处理低频配置。eBPF 侧校验 TGID、team、ABI、session、epoch 后结合就绪态、VDSQ 与 running-team 选取；接口失效时退化为无偏好轮换或 RVV Dirty 识别。组内同相位由稳定映射与 VDSQ 保证，阶段提示仅用于组间仲裁，二者解耦。

第 6 章　总 结

6.1　项目完成情况

本项目完成多实例 RISC-V LLM 推理调度的端到端实现：将 sched_ext 移植至 K1 Linux 6.6，实现 Phase-Aware Dirty Lease 调度器、loader、在线控制与 BPF maps；为 llama.cpp 增加工作线程注册与阶段提示；配套 RVV/futex 观测工具、屏障探针与可复现 Benchmark。系统支持 RVV Dirty 回退、组内互斥映射即同相位调度、分组 VDSQ、阶段优先与活性保护，卸载后恢复默认调度；另完成 x86_64 AVX2 跨架构补充移植，包含 ABI 分类、物理核级映射与阶段仲裁。补丁、源码、构建说明、脚本与报告均已归档。

6.2　核心结论与贡献

测量表明，RVV restore-elision 虽省略 93.637% 完整恢复，却无可测吞吐收益；真正瓶颈是 OpenMP 线程在逐层屏障处的到达相位。项目据此形成“组内同相位推进 + 组间阶段协调”方案：稳定映射、长向量时间片与 VDSQ 使同组线程各占一核、以相近节奏推进并同步到达 graph barrier；阶段提示优先响应 Prefill/Mixed，队列回退、受限抢占与 pending safety 维持工作守恒与活性。

屏障探针给出核心证据：8+8 共享 8 核的 Decode 阶段，到达时差中位数由 850.500 µs 降至 13.386 µs，p95 降低 95.4%。正式异步实验中，完整策略相对 CFS 在 c=2、4、8 下分别提高 TPS 54.9%、30.7% 与 25.6%，平均 TTFT 分别降低 39.0%、42.0% 与 42.2%，p95 降低 38.6%、42.3% 与 42.5%。Observe 已取得主要吞吐收益；Phase-Aware 在吞吐变化不超过 0.4% 时进一步降低 p95 TTFT，两层机制可明确区分。上下文切换减少 93.1%～96.4%，但 RVV 消融与 dynamic 对照共同表明，切换或寄存器搬运计数的下降不是吞吐提升的充分条件，收益应归因于屏障相位对齐与资源分配改善。x86 补充实验支持跨架构适用性：非超售 TTFT 降低约 41.4% 且吞吐基本保持，人工超售下吞吐恢复约 5.6 倍，XSTATE 搬运仅占 0.028%。该部分为单平台方向性验证，证据强度低于 RISC-V 正式实验。

6.3　局限与后续工作

第一，正式验证集中于 K1 八核 RVV 1.0 与 Q4_K_M 模型，线程数、内存带宽、向量长度与计算图均可能影响最优策略；后续需在更多 RISC-V 处理器、

模型规模、量化格式与实例数上重复，并加入真实到达轨迹、长短 Prompt 混合、后台负载与能耗指标。x86 部分仅在一台 4 物理核、8 逻辑 CPU 的 AVX2 平台完成有限轮次验证，未覆盖其他 SMT 拓扑、AMD 平台与 AVX-512，只能作为辅助证据。

第二，最可靠的 worker/team/phase 信息来自 patched llama.cpp。接口不可用时，RISC-V 可由 RVV Dirty 回退，x86 仅能依赖 ABI 注册；其他框架需实现相同语义 ABI，才能使用完整阶段感知，而同相位推进仅依赖身份与映射，接入要求更低。后续可将注册与提示接口整理为通用库，并探索 tracepoint、uprobe 等低侵入适配。

第三，dirty threshold、100 ms 时间片、1 s pending safety 等参数来自固定平台离线实验；短自旋对照也表明自旋与让出存在权衡。后续可根据可运行线程数、阶段比例、队列延迟与负载，进行有界、低频在线调参并保留回退；内核也可迁移至包含主线 sched_ext 的 Linux 6.12 或后续版本，重新验证 BPF kfunc、DSQ 与退出语义。

第四，同相位调度依赖实例线程数不超过核心数、且各线程能稳定占用唯一 CPU。若单实例线程数超过核心数，或存在大量抢占向量线程的系统负载，唯一映射会被压缩，组内相位难以保持，收益随之下降；此时需要限制 team 宽度，或引入准入控制与相位分组。

6.4 开源项目致谢

本项目使用 Linux、sched-ext/scx-kernel-releases、llama.cpp、libbpf、bpftool 与 elfutils 等开源项目，并遵循其版权声明与许可证。来源与引用范围见 2.3 节及补丁说明，在此感谢相关社区与开发者。

参考文献

RISC-V International. 2021. RISC-V “V” Vector Extension, Version 1.0 [EB/OL]. https://github.com/riscvarchive/riscv-v-spec/releases/tag/v1.0 [2026-07-25].

sched-ext project. 2023. scx-kernel-releases: v6.6.4-scx2 [CP/OL]. https://github.com/sched-ext/scx-kernel-releases/tree/v6.6.4-scx2 [2026-07-25].

Linux Kernel Contributors. 2024. Extensible Scheduler Class [EB/OL]. https://docs.kernel.org/scheduler/sched-ext.html [2026-07-25].

ggml-org. 2023. llama.cpp [CP/OL]. https://github.com/ggml-org/llama.cpp [2026-07-25].

SON D. 2026. Elastic Gang: Per-Token Membership Change for a Hard-Barriered LLM Inference Gang Co-Scheduled with OS Processes [EB/OL]. arXiv:2607.04668. https://doi.org/10.48550/arXiv.2607.04668 [2026-07-25].

XU Y, HAN X, LI Y, CHE W. 2026. ArcLight: A Lightweight LLM Inference Architecture for Many-Core CPUs [C]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations). Association for Computational Linguistics: 178-186. https://doi.org/10.18653/v1/2026.acl-demo.18.

ROCA A, BELTRAN V. 2026. Rethinking Thread Scheduling under Oversubscription: A User-Space Framework for Coordinating Multi-runtime and Multi-process Workloads [C]//Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming. ACM: 53-67. https://doi.org/10.1145/3774934.3786451.

GOTTSCHLAG M, BRANTSCH P, BELLOSA F. 2020. Automatic Core Specialization for AVX-512 Applications [C]//Proceedings of the 13th ACM International Systems and Storage Conference. ACM: 25-35. https://doi.org/10.1145/3383669.3398282.